Dashboard Engine Technical Documentation

Balance Sheet Exposure & Variance Analysis HTML Dashboard Generator

Prepared for Treasury Reporting process documentation

Primary source reviewed: BS EXPOSURE & VARIANCE ANALYSIS HTML DASHBOARD ENGINE - EXECUTABLE SCRIPT (1).py

Language: English

Design intent: professional PDF handover for operational and maintenance use

This documentation was prepared from the supplied dashboard engine script. It explains what the engine does, what inputs it requires, how the runtime prompts work, how the HTML dashboard is assembled, and where maintenance attention is most likely to be needed.

Contents

1. Document Purpose
2. Executive Summary
3. What the Engine Was Designed For
4. Problems Solved by the Engine
5. Required Inputs and Source Workbook Contract
6. End-to-End Processing Flow
7. Data Processing Logic
8. Dashboard Architecture
9. Page-by-Page Functional Behaviour
10. Function Reference
11. Intuition-Wise Explanation for Non-Coders
12. How to Generate a New Dashboard Output
13. Assumptions, Dependencies, Risks, and Limitations
14. Maintenance Guidance and Enhancement Backlog
15. Appendix: Required Columns and Runtime Defaults

1. Document Purpose

This document explains the design, behaviour, inputs, outputs, and maintenance logic of the Dashboard Generator script that transforms Balance Sheet Exposure and Variance Analysis BI master data into a browser-based executive dashboard. The goal is to give both business users and future maintainers a clean, end-to-end understanding of what the engine does and why it does it.

2. Executive Summary

The engine is a lightweight Python publisher. It does not calculate source treasury balances itself. Instead, it consumes one Excel workbook that already contains two curated BI tabs, reshapes those tabs into a JSON payload, embeds that payload into a prebuilt HTML/CSS/JavaScript dashboard template, saves the HTML file to the Desktop, and opens it in the default browser.

Operationally, this makes the dashboard generation step repeatable and fast. From a design perspective, the script separates concerns into four layers: input validation, data loading and transformation, HTML template generation, and browser delivery.

Key output: one local HTML file named according to the reporting period, for example BALANCE_SHEET_EXPOSURE_VARIANCE_ANALYSIS_2025_DECEMBER.html.

3. What the Engine Was Designed For

- To publish an executive-facing dashboard without requiring manual HTML editing or BI tool publishing steps.
- To consolidate two treasury reporting outputs - Balance Sheet Exposure master data and Variance Analysis master data - into one navigable dashboard experience.
- To provide a self-service delivery layer for Treasury Reporting so that a user can generate a shareable dashboard by answering only a small number of runtime prompts.
- To offer interactive analysis features such as filter bars, multi-select controls, FX sensitivity scenarios, drill-down trees, time-series views, entity pages, and variance-detail exploration.

In business terms, the engine is the presentation layer that sits on top of upstream reporting automation. It assumes the exposure and variance engines have already produced clean BI-ready data.

4. Problems Solved by the Engine

Problem	How the script addresses it
Manual dashboard preparation	Automates HTML generation from one Excel workbook, eliminating copy-paste dashboard pr
Fragmented reporting outputs	Combines exposure and variance datasets into a single browser experience with page-level r
Slow stakeholder review	Opens the finished HTML file immediately in the browser for fast review and sharing.
Weak scenario analysis	Adds runtime FX sensitivity controls so users can test how USD strengthening or weakening
Hard-to-read source data	Transforms raw BI records into visuals, KPI cards, drill-down trees, pivot tables, and time-ser

The biggest operational gain is that the reporting analyst no longer needs to handcraft a dashboard layer after producing the master data. The final delivery becomes a deterministic publication step.

5. Required Inputs and Source Workbook Contract

The engine requires exactly one source workbook at runtime. That workbook must already contain two sheets with exact sheet names: BALANCE_SHEET_EXPOSURE_BI_DATA and VARIANCE_MOVEMENTS_BI_DATA.

Balance Sheet Exposure sheet - required fields

Required field
IS_IN_ANALYSIS
_DATE_PARSED
_ENTITY
COMPANY
ACCOUNT_DESCRIPTION
TRANSACTION_CURRENCY_CODE
CURRENCY_AMOUNT
USD_AMOUNT
UNREALIZED_GAIN
UNREALIZED_LOSS
NET_CURRENCY_GAIN_LOSS

Variance Movements sheet - required fields

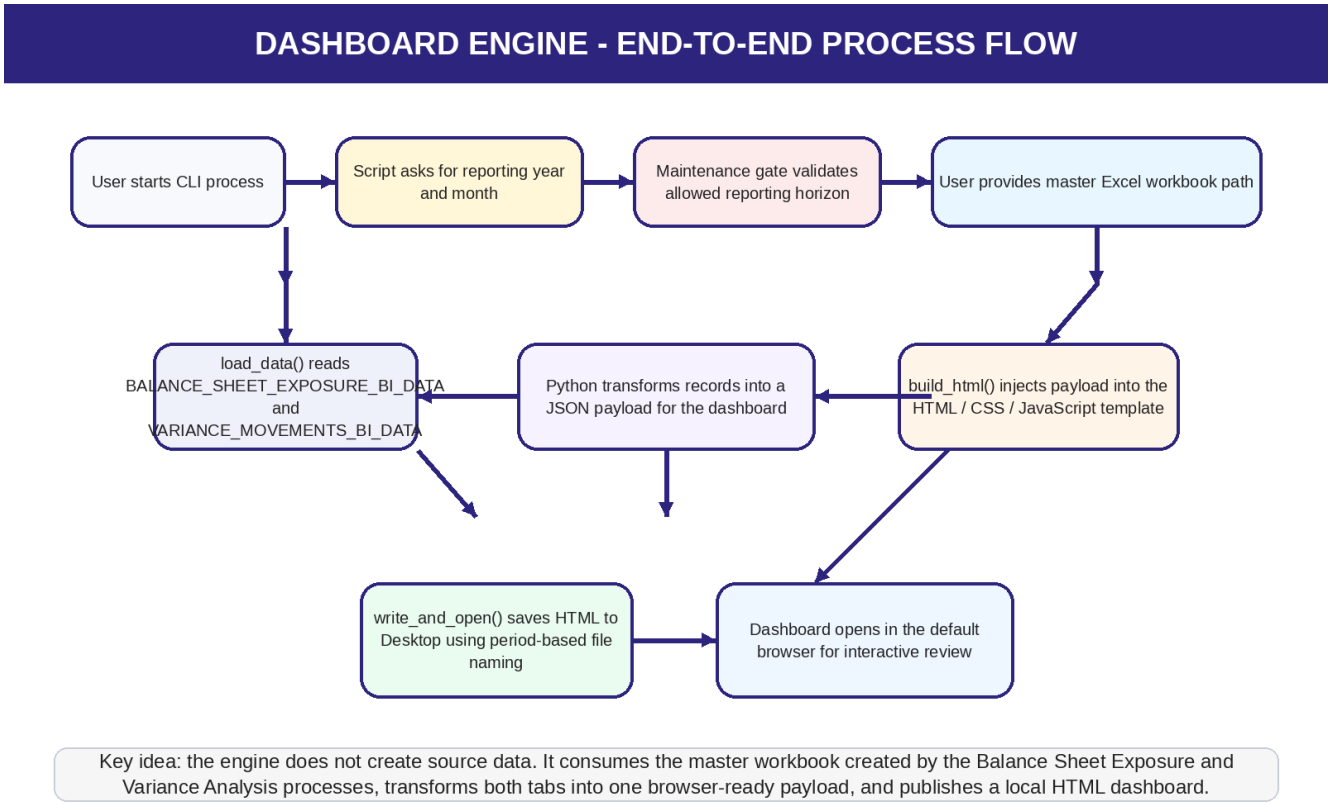
Required field
Company
Account
Account Type
Currency Code
IS_IN_ANALYSIS
Opening (t-1)
Movement (GL)
Closing (t)
Variance %
Trend
Main Driver
Month

Important source contract rules

- The Balance Sheet Exposure tab is filtered to rows where IS_IN_ANALYSIS equals CONSIDERED.
- The Variance tab keeps rows with non-empty Currency Code values and later visual logic includes CONSIDERED and NOT LABELED records while excluding NOT_CONSIDERED.
- The script expects _DATE_PARSED to already be a date-capable column because period ordering depends on it.
- The workbook is assumed to be a finished BI output, not a raw transactional export.

6. End-to-End Processing Flow

At runtime the process is intentionally short. The user starts the CLI, enters the reporting period, passes through the maintenance gate, provides the master workbook path, and then the script performs the rest automatically.



The diagram above reflects the real runtime sequence in the code: interactive prompts first, transformation second, HTML publishing third, browser delivery last.

7. Data Processing Logic

7.1 Period parsing and naming

The engine accepts flexible year-month expressions such as '2025 December', 'Dec 2025', '2025-12', and '12/2025'. It normalises the answer into year, month, and a filename-safe label such as 2025_DECEMBER. This label is later used in the final HTML filename.

7.2 Workbook ingestion

The load_data() function reads the two required sheets separately. On the exposure side, it derives a display period from _DATE_PARSED, builds an ordinal month key for sorting, and creates one flattened record per row. On the variance side, it filters out blank currency rows and builds one flattened record per row for movement analysis.

7.3 Payload construction

Instead of repeatedly reading Excel from the browser, the Python layer converts the workbook into a compact payload object. This payload contains record arrays plus metadata arrays such as periods, available currencies, accounts, companies, and the latest period. The HTML template then consumes that payload directly in JavaScript.

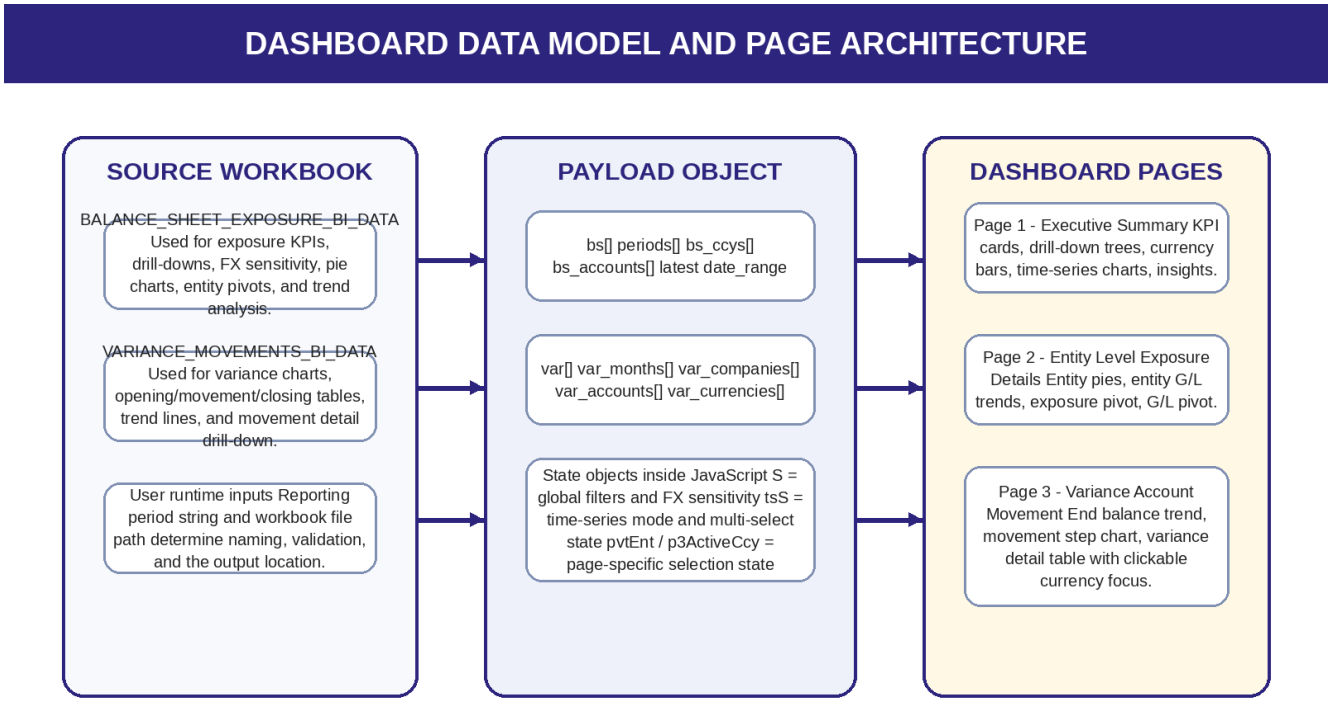
7.4 Front-end state model

Inside the browser, the dashboard uses a global state object S for period, company, currency, account, and FX sensitivity selections. Separate state objects manage time-series selections, entity pivot choices, and page-3 currency focus. This makes the HTML output interactive without requiring a backend server.

7.5 Output publication

After HTML assembly, the script writes the file to Desktop and opens it using the system default browser. On Linux it also contains browser command fallbacks if the standard webbrowser call does not work.

8. Dashboard Architecture



The architecture is intentionally simple: one source workbook, one payload, one HTML file. That simplicity is a strength because it keeps deployment friction low. The trade-off is that schema quality must be guaranteed upstream.

9. Page-by-Page Functional Behaviour

9.1 Page 1 - Executive Summary

- KPI cards for Net Exposure, Short Position, Long Position, Net Unrealized G/L, Unrealized Loss, and Unrealized Gain.
- Interactive drill-down trees for exposure and G/L by currency, then by account.
- Horizontal bar charts for exposure by currency and total unrealized G/L by currency.
- Two time-series areas with mode switching between BY ACCOUNT and BY CURRENCY plus multi-select legends.
- Narrative 'Executive Insights' cards generated directly from the currently filtered dataset.

9.2 Page 2 - Entity Level Exposure Details

- One pie chart per entity for total exposure in local currency terms.
- One unrealized G/L trend chart per entity.
- Two pivot-style tables: total exposure LCY and net currency gain/loss, both entity-switchable via header buttons.
- Hard-coded entity mapping in the script for YPOPL, YPNLBV, and YPAS (displayed as YPRODAS in labels).

9.3 Page 3 - Variance Account Movement

- An end-balance trend chart.
- A stepped movement chart for GL movement analysis.
- A detail table showing account, account type, month, currency, opening, movement, closing, variance percent, and main driver.
- Secondary axis logic for outlier currencies when their max absolute value exceeds eight times the median peer magnitude.
- Clickable chart interactions that filter the table to a selected currency until the filter is cleared.

10. Function Reference

10.1 Python functions

Function	Purpose
parse_reporting_date	Parses flexible year-month user input into normalized year, month, and filename label values.
get_desktop_path	Resolves the Desktop path across Windows, macOS, and Linux and creates the folder if it does not exist.
open_in_browser	Opens the generated local HTML file in the system default browser, with Linux fallbacks if the standard browser call f
load_data	Reads both BI sheets from Excel, filters and transforms them, then builds the JSON payload consumed by the dashb
_check_maintenance_gate	Applies a silent date-based maintenance/compliance control after the reporting period is accepted.
build_html	Injects the processed payload into the large HTML/CSS/JavaScript template and returns the final dashboard markup.
write_and_open	Writes the HTML file to disk and immediately opens it in the browser.
main	Runs the interactive CLI workflow from banner display through validation, data processing, HTML generation, and fin

10.2 JavaScript function groups

Group	Functions	Purpose
Formatting / conversion helpers	fmtK, fmtFull, fmtS, sg, pctToM, mToLbl	Convert raw numeric values and month keys into display-safe strings
FX and filtering helpers	adjU, adjG, mP, mC, mY, mA, filt, filtPrev, filtV	Apply scenario-adjusted USD calculations and global filter logic for b
Global filter UI	buildFilters, togglePeriod, toggleComp, buildMSpill, openMS, filterMS, addMSOpt, updateMS, toggleMS, renderMS, closeMS	Build the global filter bar and multi-select controls for period, co
FX scenario UI	setFX, applyCustomFX, updateFXUI	Handle preset and custom FX sensitivity scenarios and refresh scen
Master orchestration	applyAll	Acts as the main browser-side refresh controller by rerendering all re
Executive Summary rendering	renderKPIs, renderDrills, buildDrill, renderBars, renderTSCharts, renderPies	Generate KPI cards, drilldown charts, bar charts, time-series charts, a
Time-series state and controls	setTsMode, openTsDD, filterTsDD, rebuildTsDD, addTsSel, removeTsSel, toggleTsSel, selectTsOpt, closeTsDD	Manage chart mode switching, multi-select, and dropdown handling,
Page navigation	switchPage	Switches between Executive Summary, Entity Level Exposure Detai
Page 2 rendering	initP2, buildPvtSels, renderP2, renderPies, renderTrends, renderPvt	Initializes entity selectors and renders page-specific pie charts, trend

Group	Functions	Purpose
Page 3 rendering	initP3LegBtns, renderP3, renderP3Chart, clearFilterP3, renderP3Table	Builds the variance analysis page, including dual-axis outlier logic, cl
Chart lifecycle	destroyC	Safely destroys existing Chart.js objects before recreating them to av

10.3 Important implementation observations

- The docstring advertises a '--help' mode, but the actual script does not implement argument parsing; the workflow is interactive only.
- The docstring describes the dashboard as self-contained, but the HTML template still loads Google Fonts, Chart.js, and chartjs-plugin-datalabels from external CDNs.
- The build_html() function computes a reporting_label display variable, but that variable is not meaningfully surfaced in the rendered dashboard header. The header shows the payload date range instead.

11. Intuition-Wise Explanation for Non-Coders

Imagine the engine as a printing press for treasury dashboarding. The Excel workbook is the stack of approved content. The Python code is the machine that cuts, sorts, and packages that content into one deliverable. The HTML template is the publication design. The browser is the final viewing channel.

So, from a non-technical perspective, the script is not 'doing exposure accounting'. It is 'publishing exposure intelligence' from already-prepared reporting data.

12. How to Generate a New Dashboard Output

- Prepare the latest combined workbook that contains both BI tabs.
- Run the executable or Python script.
- Press Enter when prompted.
- Enter the reporting period in any accepted year-month format.
- Paste the workbook path.
- Wait for the engine to build the output and open the result in your browser.
- Review filters, KPI cards, page navigation, and chart rendering before distribution.

Expected output

A Desktop HTML file named in the form BALANCE_SHEET_EXPOSURE_VARIANCE_ANALYSIS_YYYY_MONTH.html.

13. Assumptions, Dependencies, Risks, and Limitations

Area	Observation	Operational implication
Python dependencies	Requires pandas and openpyxl.	The run will fail if these libraries are not installed.
Workbook schema	No explicit schema validation layer exists beyond missing-column access.	Missing or renamed columns will trigger read errors during processing.
External web assets	The HTML references Google Fonts and CDN JavaScript libraries.	The dashboard is locally saved but not truly air-gapped. Offline environmen
Maintenance gate	A silent cutoff is hard-coded at (2026, 10).	Future reporting periods after that cutoff may terminate unexpectedly if the c
Default filters	Initial page state defaults to latest period and YPPL-only.	First-time users may think other entities are missing unless they change the

14. Maintenance Guidance and Enhancement Backlog

- Add explicit command-line argument parsing if true CLI automation is desired.
- Embed Chart.js, the datalabels plugin, and fonts locally if a fully offline deliverable is required.
- Add workbook schema validation before load_data() so that missing columns are reported clearly.
- Move the large HTML template into a separate version-controlled template file to improve maintainability.
- Externalise hard-coded entity names and colour maps into a small configuration object.
- Consider adding logging and exception trace export similar to the Balance Sheet Exposure engine for easier support on remote user machines.
- Decide whether the maintenance gate should remain silent or be replaced with a user-readable compliance warning.

15. Appendix: Required Columns and Runtime Defaults

Topic	Value
Default selected period	Latest period found in BALANCE_SHEET_EXPOSURE_BI_DATA
Default selected company	YPOPL
Output location	Desktop
Output type	HTML dashboard
Runtime prompts	Enter key, reporting period, workbook path
Primary source tabs	BALANCE_SHEET_EXPOSURE_BI_DATA and VARIANCE_MOVEMENTS_BI_DATA